PYTHON FUNDAMENTALS

Lists & Logic

50-question practice workbook

Build stronger reasoning with list operations, conditions, strings, and small real-world data tasks.

15 EASY	**15 MEDIUM**
15 HARD	**5 COMPLEX**

What you will practise

- List operations: `append()`, `sort()`, `reverse()`, `insert()`, `remove()`, and `pop()`
- Conditional decisions, membership checks, indexing, and preserving order
- String checks and edits using `endswith()`, `capitalize()`, `replace()`, `find()`, and `count()`
- Edge cases, duplicate handling, matching positions in parallel lists, and multi-step rules

How to use this workbook

1. Read the requirements carefully and identify what must change - and what must stay unchanged.
2. Write your own code in VS Code. Use the sample as a check, not as a substitute for testing.
3. Test an ordinary case plus at least one boundary or missing-item case.
4. Mark a question complete only when you can explain why your conditions and operation order work.

No solution code, hints, or completed algorithms are included intentionally.

Scope note: this version does not require loops or unfamiliar list/string methods. Many tasks use fixed-size data so you can focus on fundamentals first; later, revisit them with loops to generalise your solutions.

Easy

Build confidence with one clear decision and one or two list operations.

E01 Reject an invalid sensor reading

Given `readings = [18, -2, 25]`, a sensor reading below zero is invalid.

Requirements: Remove the invalid reading only if it is present. Keep the order of the remaining readings unchanged.

Test: Input: `readings = [18, -2, 25]` | Output: `[18, 25]`

My logic:

E02 Place an urgent task first

A task list starts as `tasks = ["email", "report", "backup"]`. A new task is `"fix login"` and `urgent = "yes"`.

Requirements: If `urgent` is `"yes"`, place the new task at the beginning. Otherwise, add it to the end. Assume the flag is exactly `yes` or `no`.

Test: Input: `urgent = "yes"` | Output: `["fix login", "email", "report", "backup"]`

My logic:

E03 Pin a favourite without duplicating it

A playlist is `playlist = ["Orbit", "Rainy Day", "Signal"]`. The requested favourite is `"Signal"`. The favourite should appear first.

Requirements: Move the favourite to the beginning whether it is already in the playlist or not. It must appear exactly once, and every other song must keep its relative order.

Test: Input: favourite `Signal` | Output: `["Signal", "Orbit", "Rainy Day"]`

My logic:

E04 Reverse a return route

A delivery route is `route = ["Depot", "North Shop", "East Shop"]`. The driver reports `return_trip = "yes"`.

Requirements: Reverse the route only for a return trip. For any other value of the flag, keep the original route order.

Test: Input: `return_trip = "yes"` | Output: `["East Shop", "North Shop", "Depot"]`

My logic:

E05 Replace a retired shelf item safely

The shelf contains `products = ["pen", "lamp", "mug"]`. The retired product is `lamp`; its replacement is `notebook`.

Requirements: Remove the retired product if present. Add the replacement only if it is missing: place it at index `1` when the retired product was removed, or append it when the retired product was already absent. Never create a duplicate replacement.

Test: Input: the shelf above | Output: `["pen", "notebook", "mug"]`

My logic:

Easy (Continued)

Continue the same rules: be precise about order, membership, and edge cases.

E06 Show durations in the requested order

Recorded study durations are `durations = [35, 20, 45]` minutes. The display setting is `order = "longest first"`.

Requirements: Sort the durations from shortest to longest. If the display setting is `longest first`, reverse the sorted list.

Test: Input: `order = "longest first"` | Output: `[45, 35, 20]`

My logic:

E07 Repair a product label

A product label is `label = "wireless-mouse"`. The old category word is `"mouse"` and the replacement is `"device"`.

Requirements: Check whether the old word occurs in the label. Replace it only when it is present; otherwise, keep the label unchanged.

Test: Input: `wireless-mouse` | Output: `wireless-device`

My logic:

E08 Standardise three headings

A page contains `headings = ["python lists", "string practice", "logic lab!"]`.

Requirements: Create a display list with each heading capitalised. If the third original heading ends with `!`, replace that final mark with `?` in its displayed version.

Test: Input: the three headings above | Output: `["Python lists", "String practice", "Logic lab?"]`

My logic:

E09 Remove the latest draft safely

Drafts are stored in `drafts = ["intro", "methods", "final"]`. The last item is the newest draft.

Requirements: Remove and report the newest draft when the list is not empty. If the list is empty, report `No draft to remove` and do not call `pop()`.

Test: Input: `["intro", "methods", "final"]` | Output: removed `final`; remaining `["intro", "methods"]`

My logic:

E10 Replace a faulty measurement

The readings are `readings = [12, 99, 15]`. The known faulty value is `99`; its corrected value is `14`.

Requirements: If `99` is present, remove it and insert `14` at index `1`, its original position in this data. If it is absent, leave the list unchanged.

Test: Input: `[12, 99, 15]` | Output: `[12, 14, 15]`

My logic:

Easy (Continued)

Continue the same rules: be precise about order, membership, and edge cases.

E11 Remove a temporary file entry

A file list is `files = ["notes.txt", "draft.tmp", "photo.png"]`. The target entry is `"draft.tmp"`.

Requirements: Use the filename suffix to identify a temporary file. Remove the target only when it has the `.tmp` suffix and is present in the list.

Test: Input: target `draft.tmp` | Output: `["notes.txt", "photo.png"]`

My logic:

E12 Fill in a missing announcement detail

An announcement reads `notice = "Demo starts: TBD"`. The placeholder is `TBD` and the confirmed time is `Friday at 4`.

Requirements: Search for the placeholder first. Replace it only when it occurs in the announcement.

Test: Input: `Demo starts: TBD` | Output: `Demo starts: Friday at 4`

My logic:

E13 Acknowledge the first urgent task

The queue is `tasks = ["urgent", "email", "clean room"]`.

Requirements: If the first item is exactly `urgent`, remove and report that first item. Otherwise, leave the queue untouched. Do not remove a later item just because it is urgent.

Test: Input: the queue above | Output: removed `urgent`; remaining `["email", "clean room"]`

My logic:

E14 Drop one unusually low score

Three quiz scores are `scores = [72, 44, 65]`.

Requirements: Sort the scores in ascending order. Remove the first score only if it is below `50`. The list always contains exactly three scores, and at most one score should be removed.

Test: Input: `[72, 44, 65]` | Output: `[65, 72]`

My logic:

E15 Keep a rolling list of three alerts

Recent alerts are `alerts = ["login", "battery", "update"]`. A new alert is `"disk"`.

Requirements: If the new alert is missing, append it and remove the oldest alert at index `0` so the list still contains three items. If it is already present, make no changes.

Test: Input: new alert `disk` | Output: `["battery", "update", "disk"]`

My logic:

Medium

Combine list updates, conditions, strings, and more than one decision.

M16 Update a playlist in two stages

Start with `playlist = ["Intro", "Focus", "Break", "Wrap"]`. The skipped song is `"Break"`; the featured song is `"Deep Work"`.

Requirements: Remove the skipped song if present. Then, if the featured song is not already listed, insert it at index `1`. Preserve every other song and its relative order.

Test: Input: the playlist above | Output: `["Intro", "Deep Work", "Focus", "Wrap"]`

My logic:

M17 Select qualifying applicants

Three applicants are listed in matching positions: `names = ["Asha", "Ravi", "Meera"]` and `scores = [82, 49, 71]`.

Requirements: A score of `60` or higher qualifies. Build a new list of qualifying names in the original order. The data always contains exactly these three name-score pairs.

Test: Input: the lists above | Output: `["Asha", "Meera"]`

My logic:

M18 Clean a small product catalogue

Labels are `labels = ["phone-old", "tablet", "camera-old"]`.

Requirements: For each of these three fixed labels, remove the exact suffix `-old` if present, then capitalise the first character. Build a cleaned list in the same order.

Test: Input: the labels above | Output: `["Phone", "Tablet", "Camera"]`

My logic:

M19 Remove sold-out items from stock

The stock list is `stock = ["tea", "coffee", "cocoa", "juice"]`. Sold-out items are `"coffee"` and `"juice"`.

Requirements: Remove each sold-out item only if it is present. Keep all other items in their original order.

Test: Input: the stock and sold-out items above | Output: `["tea", "cocoa"]`

My logic:

M20 Merge arrivals without duplicate names

The registered list is `registered = ["Asha", "Dev"]`. Arrivals, in order, are `arrivals = ["Dev", "Mina", "Omar"]`.

Requirements: Process these three arrivals in the given order. Add a name only if it is not already in the registered list at that moment.

Test: Input: the two lists above | Output: `["Asha", "Dev", "Mina", "Omar"]`

My logic:

Medium (Continued)

Continue the same rules: be precise about order, membership, and edge cases.

M21 Detect a shared top score

Scores are `scores = [65, 92, 92, 80]`.

Requirements: Arrange scores from highest to lowest. Compare the two highest values: report `Top score shared` if they are equal; otherwise report `Single top score`.

Test: Input: the scores above | Output: `[92, 92, 80, 65]`; `Top score shared`

My logic:

M22 Clean and classify notifications

Notifications are `messages = ["server DOWN!!!", "disk full", "backup delayed!"]`.

Requirements: Create a display list by capitalising each message and replacing `!!!` with `!` wherever it occurs. Separately report whether the cleaned third message ends with `!`.

Test: Input: the messages above | Output: `["Server down!", "Disk full", "Backup delayed!"]`; important: `yes`

My logic:

M23 Cancel and reorder appointments

Appointment hours are `slots = [14, 9, 11]`. The cancelled time is `9`, and `display_order = "latest first"`.

Requirements: Remove the cancelled time if present, sort the remaining times, then reverse the result only when the display order is `latest first`.

Test: Input: the values above | Output: `[14, 11]`

My logic:

M24 Pack for the destination

The bag contains `items = ["shirt", "charger", "bottle"]`. The destination type is `"cold"`.

Requirements: For a cold destination, add `jacket` at index `1` if it is missing. For a hot destination, append `sunscreen` if it is missing. Do not add either item twice.

Test: Input: destination `cold` | Output: `["shirt", "jacket", "charger", "bottle"]`

My logic:

M25 Repair a reference code in order

A reference is `code = "INV-TBD-2026"`.

Requirements: First replace `TBD` with `A17` only if the placeholder is present. Then, if the resulting code ends with `2026`, replace that suffix with `ARCHIVE`. The order of these checks matters.

Test: Input: `INV-TBD-2026` | Output: `INV-A17-ARCHIVE`

My logic:

Medium (Continued)

Continue the same rules: be precise about order, membership, and edge cases.

M26 Update a waiting queue

The queue is `queue = ["Aman", "Riya", "Kunal"]`. `cancelled = "Aman"` and `vip = "Riya"`.

Requirements: Remove the cancelled person if present. If the VIP is present after that removal, move that person to the front without creating a duplicate. If the VIP is absent, insert the VIP at the front.

Test: Input: the queue above | Output: `["Riya", "Kunal"]`

My logic:

M27 Revise a file name and label it

A file name is `filename = "report-FINAL-draft"`.

Requirements: If `draft` occurs in the name, replace it with `review`. After the change, label the file `Temporary` only if the name ends with `.tmp`; otherwise label it `Review file`.

Test: Input: `report-FINAL-draft` | Output: `report-FINAL-review`; `Review file`

My logic:

M28 Keep product and price lists aligned

Matching positions represent products and prices: `products = ["pen", "lamp", "mug"]` and `prices = [20, 500, 150]`. The discontinued product is `lamp`, at index `1` in this data.

Requirements: If the product is present, remove it from `products` and remove the price at the same index from `prices`. The remaining product-price pairs must stay aligned.

Test: Input: the lists above | Output: `products = ["pen", "mug"]`; `prices = [20, 150]`

My logic:

M29 Compare planned and delivered items

The planned items are `planned = ["milk", "rice", "tea"]`. Delivered items are `delivered = ["rice", "tea", "sugar"]`.

Requirements: Identify which planned items are missing and which delivered items were not planned. These lists contain exactly three entries each; report the results in the original order.

Test: Input: the two lists above | Output: missing `["milk"]`; unexpected `["sugar"]`

My logic:

M30 Standardise headings and flag attention

The headings are `labels = ["python_lists", "string_methods!", "logic_lab"]`.

Requirements: Create a display list by replacing underscores with spaces and capitalising the first character of each fixed label. Also report `Attention: yes` if the original second label ends with `!`; otherwise report `Attention: no`.

Test: Input: the labels above | Output: `["Python lists", "String methods!", "Logic lab"]`; `Attention: yes`

My logic:

Hard

Stress-test ordering, validation, parallel lists, and edge cases.

H31 Repair and prioritise a shelf list

The shelf is `stock = ["pen", "eraser", "glue"]`. The damaged item is `eraser`, the new item is `marker`, and `priority_mode = "yes"`.

Requirements: Remove the damaged item if present. Add the new item only if missing, sort the shelf alphabetically, and reverse the sorted result only when priority mode is `yes`.

Test: Input: the values above | Output: `["pen", "marker", "glue"]`

My logic:

H32 Remove duplicates while preserving first appearance

Raw names are `raw = ["Ava", "Bo", "Ava", "Cy"]`.

Requirements: Build a new list containing one copy of each name, preserving the order in which each name first appears. There are exactly four entries; assess each position and do not sort the names.

Test: Input: the list above | Output: `["Ava", "Bo", "Cy"]`

My logic:

H33 Validate matching order records

Order codes and quantities align by index: `codes = ["A12", "BAD", "B07"]` and `quantities = [3, 0, 5]`.

Requirements: A record is invalid if its code is `BAD` or its quantity is zero or less. Remove every invalid record from both lists at matching positions, preserving the valid pairs.

Test: Input: the lists above | Output: `codes = ["A12", "B07"]`; `quantities = [3, 5]`

My logic:

H34 Moderate and clean three titles

Submitted titles are `titles = ["refund pending", "fake!!!", "status update"]`.

Requirements: For each fixed title, replace the word `fake` with `flagged` when found, replace `!!!` with `!` when found, and capitalise the first character of the final title. Keep the original order.

Test: Input: the titles above | Output: `["Refund pending", "Flagged!", "Status update"]`

My logic:

H35 Keep valid boundary scores

A score list is `scores = [0, 50, 100, -1]`.

Requirements: Scores from `0` through `100`, inclusive, are valid. Remove values outside that range, preserve valid boundary values, then sort valid scores in ascending order.

Test: Input: the scores above | Output: `[0, 50, 100]`

My logic:

Hard (Continued)

Continue the same rules: be precise about order, membership, and edge cases.

H36 Merge topics and pin one to the front

Existing topics are `topics = ["API", "Python", "Git"]`. New topics are `new_topics = ["Git", "SQL", "Docker"]`. The pinned topic is `SQL`.

Requirements: Check each of the three new topics in order and append only missing topics. Then, if the pinned topic is present, move it to index `0` without duplicating it. Preserve the relative order of all other topics.

Test: Input: the lists above | Output: `["SQL", "API", "Python", "Git", "Docker"]`

My logic:

H37 Clean readings but retain valid duplicates

Readings are `readings = [12, -3, 12, 5]`; negative values are invalid. The display setting is `latest_first = "yes"`.

Requirements: Remove negative readings, but keep repeated non-negative values. Sort the remaining readings ascending, then reverse only if `latest_first` is `yes`.

Test: Input: the readings above | Output: `[12, 12, 5]`

My logic:

H38 Insert a weather cover beside fragile cargo

Cargo is `boxes = ["standard-A", "fragile-B", "standard-C"]`. The weather is `rain`, and the required cover is `cover-B`.

Requirements: When the weather is rain and the fragile box is present, insert `cover-B` immediately before it, at index `1`, if the cover is not already present. Otherwise, leave the list as it is.

Test: Input: the boxes above | Output: `["standard-A", "cover-B", "fragile-B", "standard-C"]`

My logic:

H39 Filter unsafe usernames

Candidate usernames are `users = ["admin_7", "riya2026!", "team_admin", "aman2026"]`.

Requirements: Reject a username if it contains `admin` or ends with `!`. Build separate accepted and rejected lists, keeping each list in the original order. There are exactly four candidates.

Test: Input: the list above | Output: accepted `["aman2026"]`; rejected `["admin_7", "riya2026!", "team_admin"]`

My logic:

H40 Find a shared lead and a low outlier

Scores are `scores = [71, 88, 45, 88]`.

Requirements: Sort scores from highest to lowest. Report whether the highest score is shared by comparing the first two values. Also report `Low score review` if the last value is below `60`, otherwise report `No low score review`.

Test: Input: the scores above | Output: `[88, 88, 71, 45]`; `Top score shared`; `Low score review`

My logic:

Hard (Continued)

Continue the same rules: be precise about order, membership, and edge cases.

H41 Repair a short system log

The log sequence is `log = ["A-start", "B-finish", "A-start", "C-end"]`.

Requirements: The required start marker is `A-start` and the required final marker is `C-end`. If the third entry repeats the first entry, remove that third entry. If the log does not end with `C-end`, add `C-end` to the end.

Test: Input: the log above | Output: `["A-start", "B-finish", "C-end"]`

My logic:

H42 Reroute around a blocked stop

Stops are `route = ["Hub", "Zone-C", "Zone-A", "Zone-B"]`. `blocked = "Zone-C"` and `express = "yes"`.

Requirements: Remove the blocked stop if it exists. For express delivery, move `Zone-B` to the front if it is present. Keep the remaining stops in their existing relative order.

Test: Input: the values above | Output: `["Zone-B", "Hub", "Zone-A"]`

My logic:

H43 Update repeated contact details

A message is `message = "Call 555-0101 or 555-0101"`. The new number is `555-0199`.

Requirements: Check how many times the old number appears using the string `count()` method. Replace every occurrence with the new number. Report `Two references updated` only when the original count was exactly two.

Test: Input: the message above | Output: `Call 555-0199 or 555-0199`; `Two references updated`

My logic:

H44 Correct a parallel attendance record

Names and statuses align by index: `names = ["Isha", "Nikhil", "Pari"]` and `status = ["present", "absent", "present"]`. A replacement attendee is `Zara`.

Requirements: Remove each absent attendee and the matching status entry. Then insert `Zara` with status `present` at index `1`. The provided data contains exactly one absent attendee.

Test: Input: the two lists above | Output: `names = ["Isha", "Zara", "Pari"]`; `status = ["present", "present", "present"]`

My logic:

H45 Prepare a task list with an exception

Tasks are `tasks = ["write", "review", "deploy"]`. The blocked task is `review`, the new task is `test`, and `priority = "yes"`.

Requirements: Remove the blocked task if present. Add `test` only if it is missing. If priority is `yes`, move `deploy` to the front when it is present. Do not duplicate a task during the update.

Test: Input: the values above | Output: `["deploy", "write", "test"]`

My logic:

Complex Reasoning & Real-World Problem-Solving

Plan a small workflow, respect record relationships, and apply rules in the correct order.

C46 Audit student records before classifying results

Four records are stored in matching positions: `names = ["Asha", "Kabir", "Mina", "Zoya"]` and `marks = [78, -1, 92, 35]`.

Requirements: Marks outside `0` to `100` are invalid; remove the name and mark from the same position. Of the remaining records, a mark of `40` or more is a pass. Build separate lists of passed names and failed names. The dataset always has these four records.

Test: Input: the lists above | Output: passed `["Asha", "Mina"]`; failed `["Zoya"]`

My logic:

C47 Reconcile stock and preserve product-quantity pairs

Products are `products = ["tea", "coffee", "rice", "soap"]`; matching quantities are `qty = [0, 5, -2, 8]`. A new product is `salt` with quantity `12`; `priority_product = "soap"`.

Requirements: First remove records with negative quantities, then remove records with zero quantity. Keep product and quantity indices aligned at every stage. Add salt only if missing. Finally, move the priority product and its quantity to index `0` if that product is present.

Test: Input: the lists above | Output: `products = ["soap", "coffee", "salt"]`; `qty = [8, 5, 12]`

My logic:

C48 Combine attendance lists and apply final corrections

Morning attendance is `morning = ["Riya", "Aman", "Leena"]`. Afternoon arrivals are `afternoon = ["Aman", "Nikhil", "Riya"]`. Leena has withdrawn and Nikhil is the VIP attendee.

Requirements: Start with the morning list. Check the three afternoon arrivals in order and add only names not already present. Remove Leena if present. If Nikhil is present, move Nikhil to the front without duplication.

Test: Input: the lists above | Output: `["Nikhil", "Riya", "Aman"]`

My logic:

Complex Reasoning & Real-World Problem-Solving (Continued)

Continue the same rules: be precise about order, membership, and edge cases.

C49 Validate a small transaction ledger

Transaction amounts are `ledger = [500, -80, -120, 0]`. A valid transaction is positive, or is a refund from `-100` through `-1`. Values below `-100` and zero are invalid.

Requirements: Build valid and rejected lists in their original order. Calculate the net total of valid transactions. The supplied ledger always contains these four amounts, so you can evaluate the entries individually.

Test: Input: the ledger above | Output: valid `[500, -80]`; rejected `[-120, 0]`; net total `420`

My logic:

C50 Update a support queue without breaking priority rules

The queue is `queue = ["T-12:normal", "T-07:critical", "T-22:high"]`. The resolved ticket is `T-12:normal`; the new ticket is `T-31:critical`; `maintenance_mode = "no"`.

Requirements: Apply these rules in order: remove the resolved ticket if present; do not add a ticket that is already in the queue; if the new ticket is missing and ends with `:critical`, insert it at the front, otherwise append it; finally reverse the full queue only when maintenance mode is `yes`.

Test: Input: the queue and settings above | Output: `["T-31:critical", "T-07:critical", "T-22:high"]`

My logic:

Practice Tracker

Mark a question complete only when you can solve it and explain your reasoning independently.

Use R for revised, H for solved with help, and I for solved independently.

Question	Status	Date	Question	Status	Date
[ ] E01	_____	_____	[ ] M26	_____	_____
[ ] E02	_____	_____	[ ] M27	_____	_____
[ ] E03	_____	_____	[ ] M28	_____	_____
[ ] E04	_____	_____	[ ] M29	_____	_____
[ ] E05	_____	_____	[ ] M30	_____	_____
[ ] E06	_____	_____	[ ] H31	_____	_____
[ ] E07	_____	_____	[ ] H32	_____	_____
[ ] E08	_____	_____	[ ] H33	_____	_____
[ ] E09	_____	_____	[ ] H34	_____	_____
[ ] E10	_____	_____	[ ] H35	_____	_____
[ ] E11	_____	_____	[ ] H36	_____	_____
[ ] E12	_____	_____	[ ] H37	_____	_____
[ ] E13	_____	_____	[ ] H38	_____	_____
[ ] E14	_____	_____	[ ] H39	_____	_____
[ ] E15	_____	_____	[ ] H40	_____	_____
[ ] M16	_____	_____	[ ] H41	_____	_____
[ ] M17	_____	_____	[ ] H42	_____	_____
[ ] M18	_____	_____	[ ] H43	_____	_____
[ ] M19	_____	_____	[ ] H44	_____	_____
[ ] M20	_____	_____	[ ] H45	_____	_____
[ ] M21	_____	_____	[ ] C46	_____	_____
[ ] M22	_____	_____	[ ] C47	_____	_____
[ ] M23	_____	_____	[ ] C48	_____	_____
[ ] M24	_____	_____	[ ] C49	_____	_____
[ ] M25	_____	_____	[ ] C50	_____	_____

Quick self-check before moving on

[ ] I can explain what each condition checks.
[ ] I can predict how an operation changes the list and its indices.
[ ] I tested missing items, duplicates, and boundary values where relevant.
[ ] I can reproduce the solution without copying it from notes.